Imitation Learning

자율주행 이론 및 실습
11/05

Hosung Lee, Sangjin Han, Jiwon Kong,
Seunggu Kang, Hyunho Song, Hyeongjun Song



Contents

RLLAB
http://rllab.snu.ac.kr

- **Final Project**
  - Remaining schedule
  - Rules
- **Imitation Learning**
  - About imitation learning
  - Codes
  - Example videos

Final Project

RLLAB
http://rllab.snu.ac.kr

- Remaining schedule (https://rllab.snu.ac.kr/courses/autonomous-driving-2026)

- Week 9
  - 10/27: Camera technologies for autonomous driving
  - 10/29: Lab - Introduction to F1TENTH simulator
- Week 10
  - 11/03: Markov decision processes, Reinforcement learning (RL), Behavior cloning
  - 11/05: Lab - RC Car Calibration
- Week 11
  - 11/10: Soft MDP, Sparse MDP, DQN
  - 11/12: Lab - Imitation Learning (Simulation)
- Week 12
  - 11/17: TRPO, PPO, Maximum entropy RL
  - 11/19: Lab - Imitation Learning (Real)
- Week 13
  - 11/24: Inverse RL (IRL), Maximum entropy IRL
  - 11/26: Lab - Reinforcement Learning 1: PPO / SAC (Simulation)
- Week 14
  - 12/01: Survey: Autonomous driving | Lab - Reinforcement Learning (Real)
  - 12/03: Lab - Reinforcement Learning (Real)
- Week 15
- Week 16
  - 12/15: Racing Contest (Qualifying)
  - 12/17: Racing Contest (Final)

← Today

- About 5 weeks left

- Time to start preparing for the racing day

Final Project

RLLAB
http://rllab.snu.ac.kr

- Remaining schedule (for Lab session)

11/05: Lab - Imitation Learning ← Today

11/10: Lab - Sim2Real
11/12: Lab - Reinforcement Learning (PPO, SAC)

11/17:교수님 수업
11/19:교수님 수업

12/15: Racing contest (Qualifying)
12/17: Racing contest (Final)

- About 5 weeks left

- Time to start preparing for the racing day

Final Project

RLLAB
http://rllab.snu.ac.kr

- Rules (Can be updated)

  - The tracks are scheduled to be installed in a larger form behind Room 401 in Building 300.

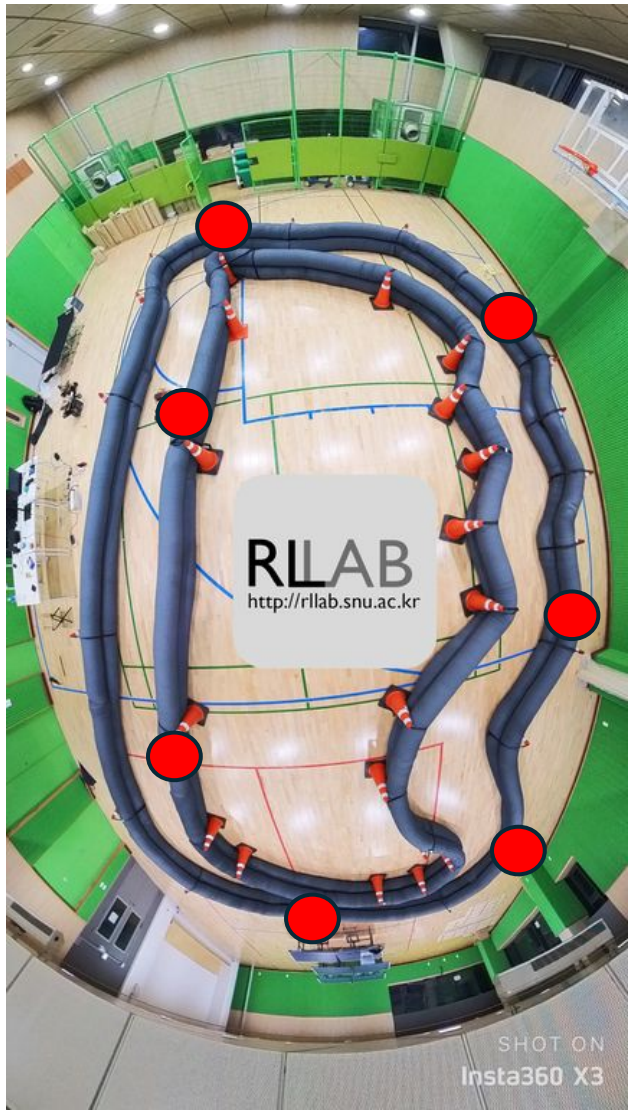
  - The track on the second day of racing is different from the one on the first day.

  - Each team will have two inference opportunities each day.

  - The priority is based on the number of waypoints completed. If teams complete the same number of waypoints, the team that took less time will receive a higher score.

  - There are no restrictions on the model as long as the input and output are the same.

  - There are no restrictions on the RC car configuration as long as no additional parts are used.



: waypoints

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

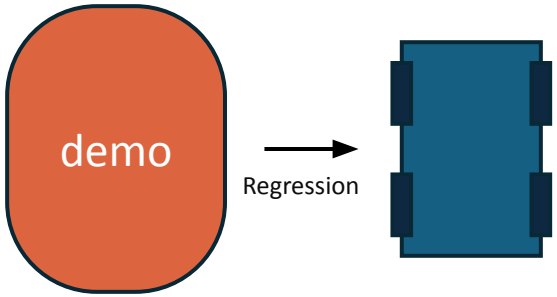
- Imitation Learning
  - A method where an agent learns by using demonstrations provided to it.
  - Includes behavior cloning (BC) and inverse reinforcement learning (IRL).

BC

- Regression problem
- Various methods for regression can be used, e.g. Gaussian process regression (GPR), deep learning…

IRL

- Learns reward function from demonstration
- Train agent with obtained reward with RL.

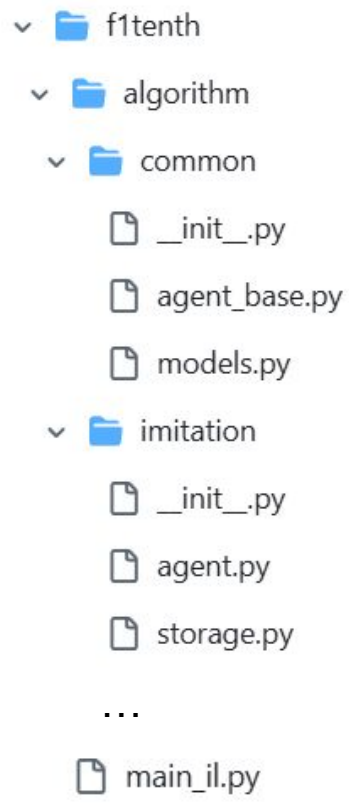




Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Codes



- common/
  - agent_base.py Abstract class
  - ***models.py** NN models
- imitation/
  - ***agent.py** Imitation learning agent
  - storage.py Store state and action demo
- *main_il.py* python execution code

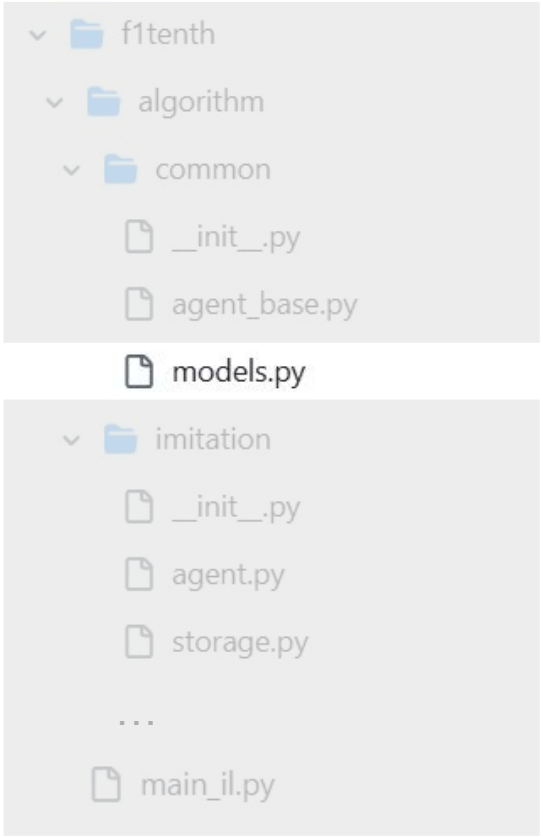
  (collecting demo + training agent)

```
git clone https://github.com/jwonkong/Autonomous-Driving-2026.git ros2_ws
```

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Codes




```
class Actor(nn.Module):
    def __init__(self, args):
        ...
        # ======= implement here ======= #
        self.fc1 =
        self.fc2 =

        self.fc_mean =
        # ============================== #
        ...
    def forward(self, x):
        # ======= implement here ======= #
        x =
        # ============================== #
        mean = self.output_activ(self.fc_mean(x))

        log_std = torch.clamp(self.log_std, min=LOG_STD_MIN, max=LOG_STD_MAX)
        std = torch.ones_like(mean)*torch.exp(log_std)

        action_dists = torch.distributions.Normal(mean, std)
        return action_dists
```

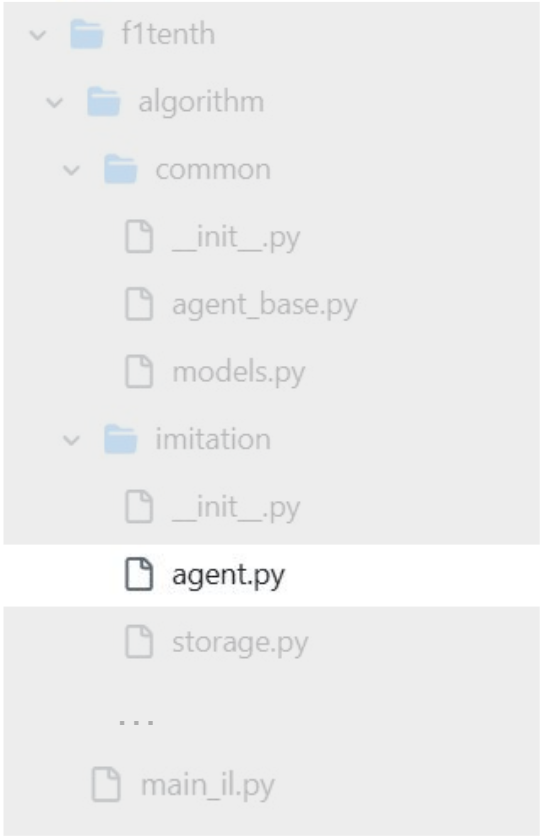
☐ Define your model

☐ Implement forward fn

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Codes




```
class Agent(AgentBase):

    ...


    def train(self):
        states_tensor, actions_tensor = self.rollout_dataset.getBatches()
        action_dists = self.actor(states_tensor)

        # ============================ implement here ============================ #
        MSE_Loss =
        action_mean =
        actor_loss =
        # ======================================================================== #

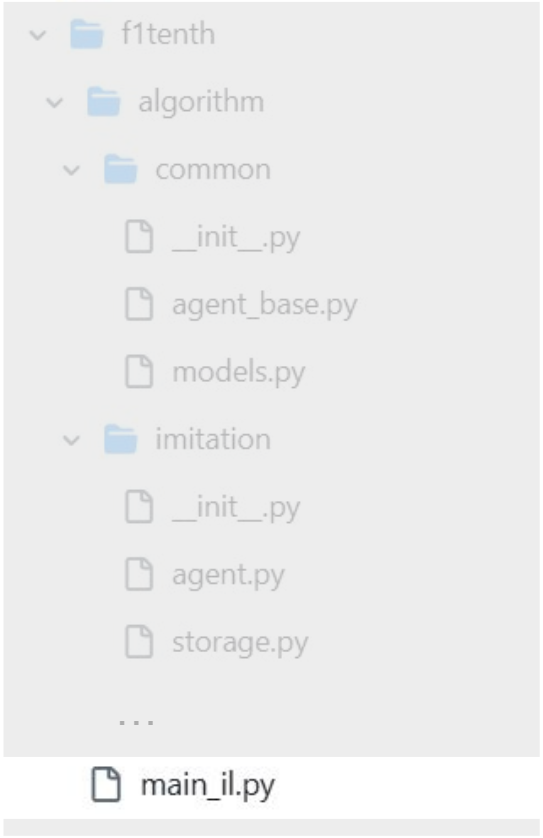
        ...
```

- Define your loss
  (objective to mimimize)

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Codes




```
def getParser():
```

- get arguments when running code

```
def eval(args, env, agent, name, eval_num=1):
```

- evaluate agent performance

```
def train(args):
```

- train agent

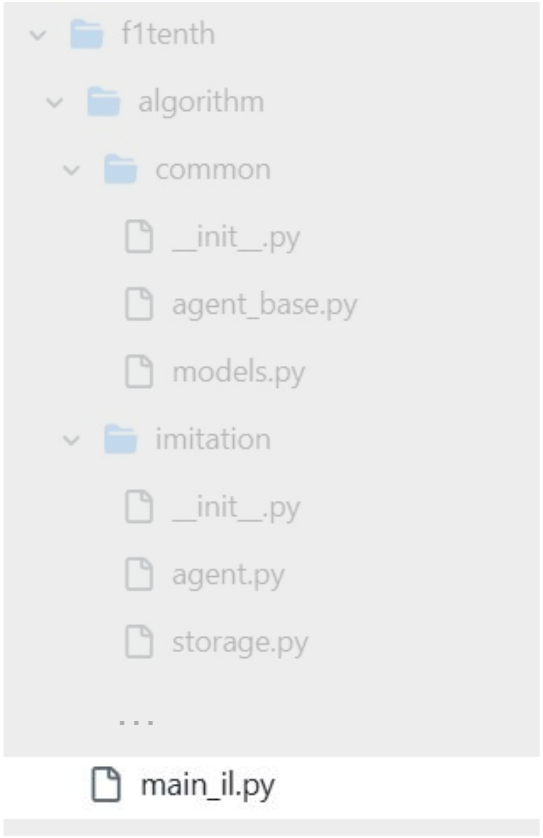
```
def test(args):
```

- test agent (inference trained model)

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Codes



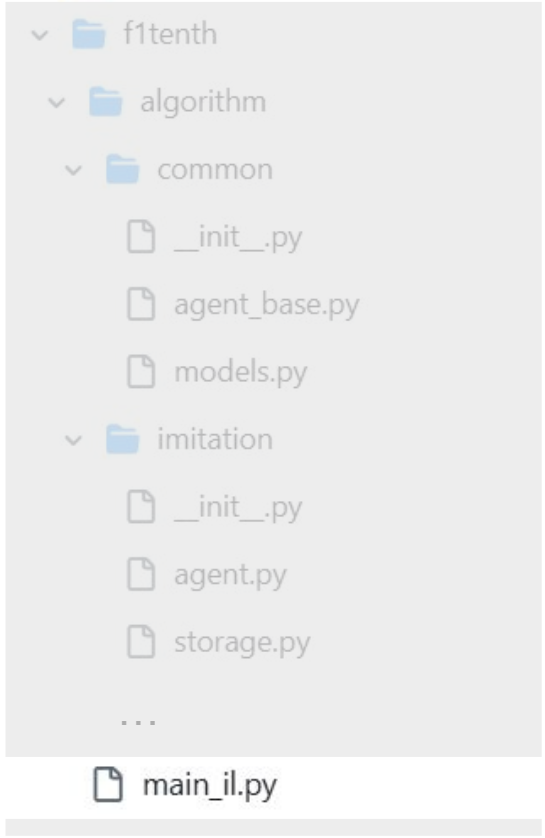

```python
def getParser():
    parser = argparse.ArgumentParser(description='F1tenth.')
    # mode
    parser.add_argument('--test',  action='store_true', help='for test.')
    # algorithm
    parser.add_argument('--name', type=str, default=None, help='algo name.')
    parser.add_argument('--algo_idx', type=int, default=1, help='algo index.')
    parser.add_argument('--model_num', type=int, default=None, help='for checkpoint num.')
    # common
    parser.add_argument('--wandb',  action='store_true', help='use wandb?')
    parser.add_argument('--comment', type=str, default=None, help='wandb comment saved in run name.')
    parser.add_argument('--device', type=str, default='gpu', help='gpu or cpu.')
    parser.add_argument('--gpu_idx', type=int, default=0, help='GPU index.')
    parser.add_argument('--seed', type=int, default=1, help='seed number.')
    parser.add_argument('--log_freq', type=int, default=int(1e4), help='# of time steps for logging.')
    parser.add_argument('--save_freq', type=int, default=int(5e5), help='# of time steps for save.')
    # train
    parser.add_argument('--epochs', type=int, default=50, help='# of epochs for update.')
    parser.add_argument('--threshold', type=int, default=200, help='minimum steps of expert rollout.')
    parser.add_argument('--total_steps', type=int, default=int(1e5), help='total interaction steps.')
    # for eval
    parser.add_argument('--eval_num', type=int, default=5, help='# of evaluation epochs.')
    parser.add_argument('--eval_freq', type=int, default=int(5e4), help='# of time steps for eval.')
    parser.add_argument('--save_video', action='store_true', help='save video.')
    parser.add_argument('--live_video', action='store_true', help='live video.')
    return parser
```

usage example:

```
(f1tenth) jun@rllab2:~/projects/f1tenth$ python main_il.py --name bc --algo_idx 1 --save_video
```

Imitation Learning

- Codes




```
def train(args):
```

```
# ======= collect trajectories ======= #
expert_data_path = 'expert_data.pkl'
if os.path.isfile(expert_data_path):
    cprint(f'\n[Rollout] load.', bold=True, underline=True)
    with open(expert_data_path, 'rb') as f:
        expert_data = pickle.load(f)
        observations = expert_data['observations']
        actions = expert_data['actions']
else:
    cprint(f'\n[Rollout] start.', bold=True, underline=True)
    total_step = 0
    observations, actions = [], []
    while total_step < args.total_steps:

        observation, info = train_env.reset()
        track = train_env._env.unwrapped.track
        planner = PurePursuitPlanner(track=track, wb=0.17145 + 0.15875)
        ep_observations, ep_actions = [], []
        ep_step, ep_reward = 0, 0

        terminate = False
        while ep_step < args.max_episode_steps and not terminate:
            speed, steer = planner.plan(info['obs_dict'], PARAM)
            action = np.clip([steer/(train_env.max_steer+EPS), speed/train_env.max_speed], -np.ones(2), np.ones(2))
            ep_observations.append(observation)
            ep_actions.append(action)

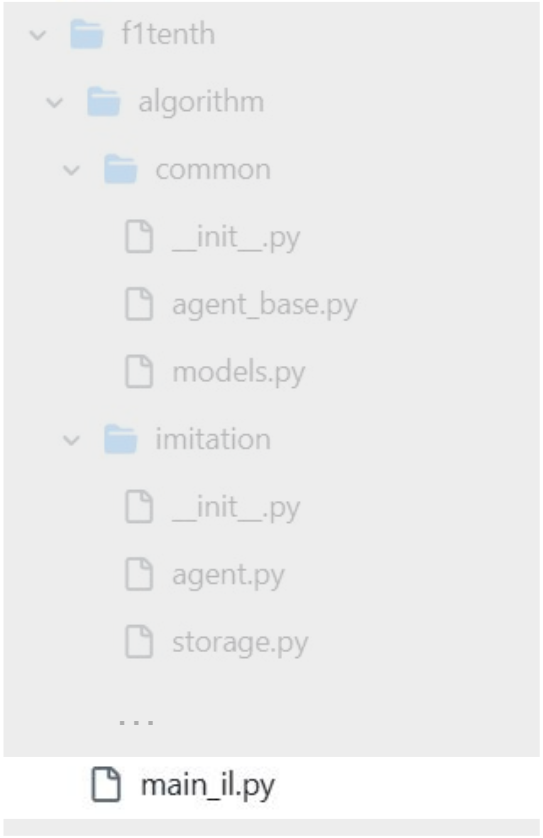
            observation, reward, terminate, truncate, info = train_env.step(action)
            ep_step += 1
            ep_reward += reward

        if ep_step > args.threshold:
            observations.append(ep_observations)
            actions.append(ep_actions)
            total_step += ep_step
            print(f'[Total step] {total_step}: {ep_step}/{ep_reward:.2f}')

    observations = np.concatenate(observations, axis=0)
    actions = np.concatenate(actions, axis=0)
    with open('expert_data.pkl', 'wb') as f:
        pickle.dump({'observations':observations, 'actions':actions}, f)
agent.step(observations, actions)
train_env.close()
# ================================== #
```

Imitation Learning

- Codes




```
def train(args):
```

```
# ======= collect trajectories ======= #
expert_data_path = 'expert_data.pkl'
if os.path.isfile(expert_data_path):
    cprint(f'\n[Rollout] load.', bold=True, underline=True)
    with open(expert_data_path, 'rb') as f:
        expert_data = pickle.load(f)
        observations = expert_data['observations']
        actions = expert_data['actions']
else:
    cprint(f'\n[Rollout] start.', bold=True, underline=True)
    total_step = 0
    observations, actions = [], []
    while total_step < args.total_steps:

        observation, info = train_env.reset()
        track = train_env._env.unwrapped.track
        planner = PurePursuitPlanner(track=track, wb=0.17145 + 0.15875)
        ep_observations, ep_actions = [], []
```

1. nearest_point_on_trajectory
   - Find nearest point on trajectory $t = \frac{(P - A) \cdot (B - A)}{||B - A||^2}$ $\text{projection} = A + t \cdot (B - A)$

2. first_point_on_trajectory_interesting_circle
   - Find the first point on the trajectory that is at a distance of 'radius' from the given point by solving 2nd order equation:

$$a = V \cdot V, \quad b = 2 \cdot V \cdot (S - P), \quad c = S \cdot S + P \cdot P - 2 \cdot S \cdot P - r^2$$

where *S* is the starting point, *P* is the given point, *V* is the vector from the starting point to the endpoint, and *r* is the radius.

3. get_actuation
   - Calculate the speed and steering angle based on the given lookahead_point

$$\text{waypoint}_y = \sin(-\theta) \cdot (x_{\text{goal}} - x_{\text{position}}) + \cos(-\theta) \cdot (y_{\text{goal}} - y_{\text{position}})$$

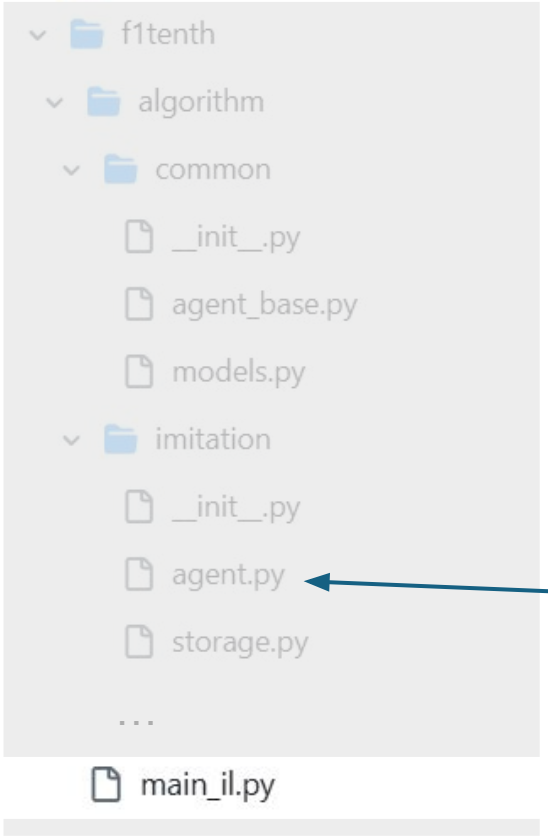
$$\text{radius} = \frac{\text{lookahead_distance}^2}{2 \times \text{waypoint}_y}$$

$$\text{speed} = \text{vgain} \times \text{lookahead_point}[2] \quad \text{steering angle} = \arctan\left(\frac{\text{wheelbase}}{\text{radius}}\right)$$

```
# ================================== #
```

Imitation Learning

- Codes




```python
def train(args):
    # ======= train ======= #
    best_score = -np.inf
    cprint(f'\n[Train] start.', bold=True, underline=True)
    for epoch in range(1, args.epochs+1):
        total_step = epoch*args.batch_size
        train_results = agent.train()
```

recall) agent.train() in agent.py

```python
class Agent(AgentBase):

    ...

    def train(self):
        states_tensor, actions_tensor = self.rollout_dataset.getBatches()
        action_dists = self.actor(states_tensor)

        # ============================ implement here ============================ #
        MSE_Loss =
        action_mean =
        actor_loss =
        # ======================================================================== #

        ...
```

← Define your loss
(objective to mimimize)

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Example video

```
(f1tenth) jun@rllab2:~/projects/f1tenth$ python main_il.py --name bc --algo_idx 1 --total_steps 100000 --save_video
[torch] cuda is used.
[load] agent_obs fail.
[load] agent_reward fail.
[bc] load fail.

[Rollout] start.
[Total step] 1000: 1000/4897.71
[Total step] 2000: 1000/4996.06
[Total step] 2210: 210/945.67
[Total step] 3210: 1000/4893.00
```

□ collecting expert demonstration

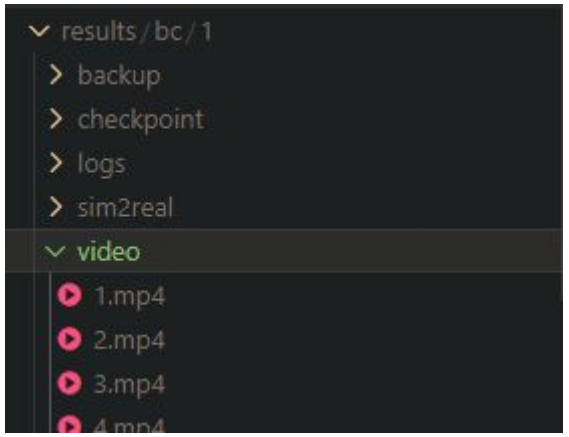
```
[Total step] 98270: 1000/4990.37
[Total step] 99270: 1000/5030.84
[Total step] 100270: 1000/4919.22

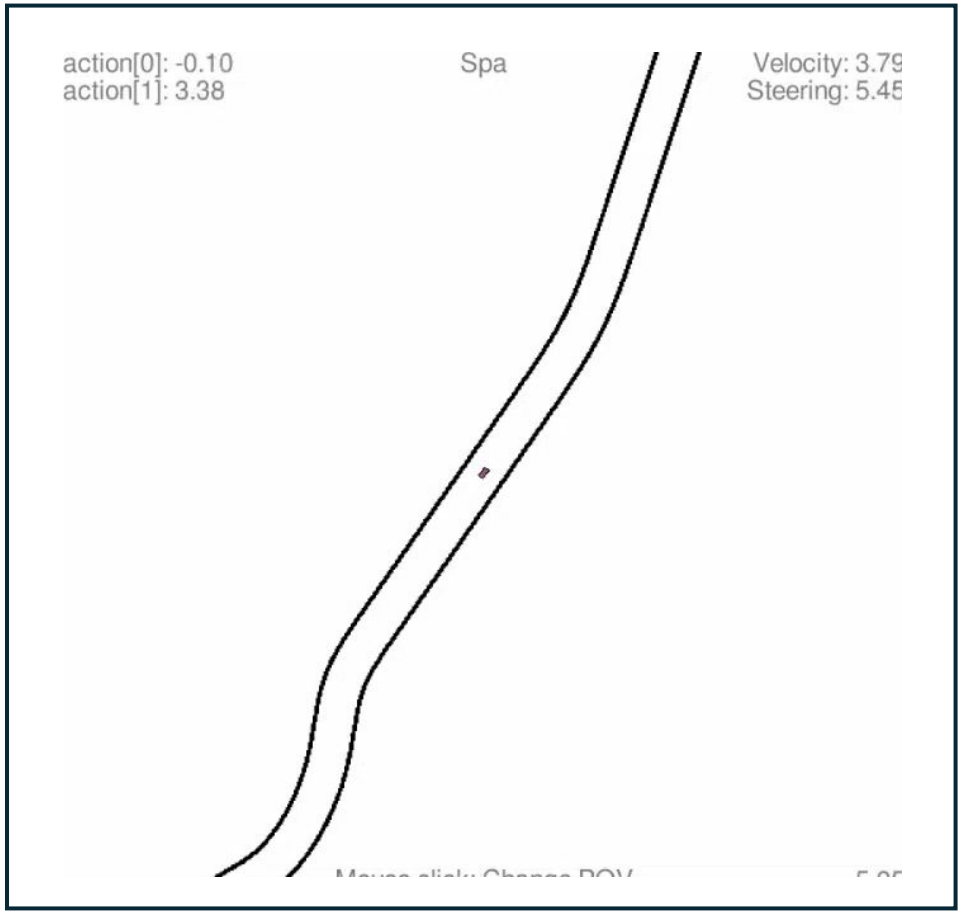
[Train] start.
[Evaluation]          Map       Lap Time
[Fail]            Catalunya    25.02
[Fail]               Spa        6.00
[Fail]               IMS       25.02
[Fail]            YasMarina    19.22
[Fail]               Spa        2.55
[Eval]-1        steps: 621.60   score: 2393.84
[bc] save best model.
{'rollout/step': 1000, 'train/best': 1, 'rollout/score': 2393.8446723842153, 'rollout/ep_len': 621.6, 'train/actor_loss': 0.5178298354148865}
```

Imitation Learning

RLLAB
http://rllab.snu.ac.kr

- Example video





Good Luck!